Bài thực hành 3 - Task 1

Chuẩn bị kiến thức cho hệ thống đại lý bảo hiểm trên .NET microservices

Sinh viên: Trương Đắc Kiên
Mã sinh viên: 2424802010029
Lớp: D24CNTT05
GitHub: KienTD204
Ngày hoàn thiện: 11/10/2026

Phạm vi

Trả lời hai câu hỏi cho đủ 21 đối tượng của đề: đối tượng là gì và có chức năng gì. Phần liên hệ với source phân biệt kiến thức kiến trúc với công nghệ thực sự có trong snapshot được dùng cho bài thực hành.

1. Các đối tượng nghiên cứu

1. CQRS

Là gì? Mẫu kiến trúc tách mô hình xử lý lệnh thay đổi trạng thái (Command) khỏi mô hình truy vấn chỉ đọc (Query). Hai phía có thể dùng chung một cơ sở dữ liệu hoặc có mô hình lưu trữ khác nhau.

Chức năng và liên hệ: Giúp mỗi phía tối ưu theo nhu cầu riêng, tách kiểm tra nghiệp vụ khi ghi với việc tổng hợp dữ liệu khi đọc. Trong source, command/query qua MediatR là cách tổ chức xử lý; riêng sự hiện diện của MediatR chưa chứng minh toàn bộ hệ thống dùng hai database độc lập.

Nguồn: https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs

2. Event Sourcing

Là gì? Cách lưu trạng thái nghiệp vụ bằng chuỗi sự kiện bất biến theo thứ tự, thay vì chỉ lưu ảnh trạng thái cuối cùng. Trạng thái hiện tại được xây lại bằng cách áp dụng các sự kiện.

Chức năng và liên hệ: Hỗ trợ truy vết lịch sử, tái dựng trạng thái và xây projection để truy vấn. Cần quản lý thứ tự, phiên bản và tính nhất quán. Source có phát sự kiện RabbitMQ nhưng điều đó không đồng nghĩa đã triển khai Event Sourcing.

Nguồn: https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing

3. Entity Framework Core

Là gì? Thư viện ORM của .NET, ánh xạ entity và quan hệ sang dữ liệu, cung cấp DbContext để truy vấn và lưu thay đổi. Hỗ trợ nhiều database thông qua provider.

Chức năng và liên hệ: Giảm mã truy cập dữ liệu, hỗ trợ LINQ, theo dõi thay đổi và migration. ProductService của snapshot dùng EF Core với cấu hình in-memory cho dữ liệu sản phẩm demo; nó không tạo thêm một database PostgreSQL sản phẩm trong bốn database được yêu cầu.

Nguồn: https://learn.microsoft.com/en-us/ef/core/

4. PostgreSQL

Là gì? Hệ quản trị cơ sở dữ liệu quan hệ đối tượng, hỗ trợ SQL, giao dịch, ràng buộc và các kiểu dữ liệu mở rộng như JSON.

Chức năng và liên hệ: Lưu dữ liệu bền vững cho Policy, Pricing, Payment và Hangfire. Bài thực hành tạo bốn database riêng. SQL ban đầu chỉ tạo role/database/quyền, còn bảng được thư viện và ứng dụng tạo khi chạy.

Nguồn: https://www.postgresql.org/docs/16/intro-whatis.html

5. NHibernate

Là gì? ORM cho .NET, ánh xạ mô hình đối tượng sang bảng quan hệ; dùng session để thực hiện truy vấn và quản lý giao dịch.

Chức năng và liên hệ: Giúp mô hình miền tách khỏi mã SQL trực tiếp, hỗ trợ mapping, lazy loading và kiểm soát persistence. PolicyService dùng NHibernate và các tệp mapping để lưu đề nghị, hợp đồng cùng outbox.

Nguồn: https://nhibernate.info/doc/nhibernate-reference/

6. Marten

Là gì? Thư viện .NET khai thác PostgreSQL để cung cấp document database và event store. Có thể lưu đối tượng thành JSON và truy vấn bằng API .NET.

Chức năng và liên hệ: Cho phép lưu mô hình linh hoạt, quản lý document session và hỗ trợ event sourcing khi ứng dụng sử dụng chức năng tương ứng. Trong snapshot, Pricing và Payment dùng document storage; không suy ra cả hai đã dùng event store chỉ vì có Marten.

Nguồn: https://martendb.io/

7. NEST

Là gì? Client .NET cấp cao của Elasticsearch thuộc dòng 7.x, cung cấp API có kiểu để xây truy vấn, index và ánh xạ dữ liệu.

Chức năng và liên hệ: Giúp ứng dụng .NET giao tiếp Elasticsearch. Đề dùng tên NEST, nhưng source .NET 8 được kiểm tra sử dụng Elastic.Clients.Elasticsearch 8.9.3; không cài thêm NEST chỉ để khớp tên trong tài liệu cũ.

Nguồn: https://www.elastic.co/guide/en/elasticsearch/client/net-api/7.17/nest.html

8. Elasticsearch

Là gì? Công cụ tìm kiếm và phân tích phân tán, lưu document trong index và hỗ trợ truy vấn toàn văn, lọc, tổng hợp.

Chức năng và liên hệ: Tạo mô hình đọc cho tìm kiếm hợp đồng và thống kê bán hàng. PolicySearch và Dashboard nhận sự kiện để cập nhật index, vì vậy dữ liệu đọc có thể xuất hiện sau khi giao dịch ghi hoàn thành.

Nguồn: https://www.elastic.co/docs/solutions/search

9. HTTP REST

Là gì? REST là phong cách kiến trúc cho hệ thống phân tán, thường biểu diễn tài nguyên qua URI và dùng các phương thức HTTP. Một API HTTP chỉ dùng JSON chưa đủ để chứng minh đáp ứng mọi ràng buộc REST.

Chức năng và liên hệ: Cung cấp giao tiếp đồng bộ giữa frontend, gateway và dịch vụ. GET đọc tài nguyên; POST thường tạo hoặc thực hiện tác vụ. Policy gọi Pricing đồng bộ để lấy phí trước khi lưu báo giá.

Nguồn: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

10. RabbitMQ

Là gì? Message broker giúp producer và consumer giao tiếp qua exchange, binding và queue, hỗ trợ xác nhận và định tuyến thông điệp.

Chức năng và liên hệ: Tách các dịch vụ khi xử lý bất đồng bộ. PolicyCreated được Payment, Search, Dashboard và Chat nhận để cập nhật phần việc của mình. Consumer cần xử lý lỗi và tránh tạo dữ liệu trùng khi thông điệp được giao lại.

Nguồn: https://www.rabbitmq.com/tutorials

11. Ocelot

Là gì? Thư viện API Gateway cho .NET, xử lý định tuyến các yêu cầu từ phía ngoài đến dịch vụ phía sau theo cấu hình.

Chức năng và liên hệ: Tập trung routing, xác thực và một số chính sách như cache. AgentPortalApiGateway kiểm tra JWT cùng claim SALESMAN và dùng Eureka để tìm dịch vụ theo tên. Gateway không thay thế kiểm tra nghiệp vụ bên trong service.

Nguồn: https://ocelot.readthedocs.io/en/latest/introduction/gettingstarted.html

12. Eureka

Là gì? Service registry của Netflix, lưu thông tin các instance dịch vụ đăng ký và trạng thái của chúng. Client có thể tra registry để tìm địa chỉ dịch vụ.

Chức năng và liên hệ: Hỗ trợ service discovery, heartbeat và tìm instance theo tên, giúp giảm phụ thuộc vào địa chỉ cố định giữa các service. Source dùng một Eureka server Java và các client .NET Steeltoe.

Nguồn: https://github.com/Netflix/eureka

13. Polly

Là gì? Thư viện tăng khả năng chịu lỗi cho .NET, cung cấp các chiến lược như retry, timeout và circuit breaker.

Chức năng và liên hệ: Giúp xử lý lỗi tạm thời ở lời gọi ra ngoài. Retry phải cân nhắc tính idempotent để tránh tạo giao dịch trùng; timeout và circuit breaker giúp giới hạn thời gian chờ và chuỗi lỗi lan truyền.

Nguồn: https://www.pollydocs.org/

14. Serilog

Là gì? Thư viện logging .NET theo sự kiện có cấu trúc; dữ liệu log gồm message template và các thuộc tính có tên.

Chức năng và liên hệ: Hỗ trợ lọc, tìm và đối chiếu log giữa các service; có thể gửi log qua các sink khác nhau. Source Pricing và Policy dùng Serilog để ghi tiến trình host và lỗi. Không ghi token hay mật khẩu thật vào log.

Nguồn: https://serilog.net/

15. JWT

Là gì? Định dạng token biểu diễn tập claim, được chuẩn hóa trong RFC 7519. Token có thể được bảo vệ bằng chữ ký hoặc MAC; payload của token ký không tự được mã hóa.

Chức năng và liên hệ: Truyền thông tin danh tính/quyền giữa client và server. Auth cấp token; gateway kiểm tra tính hợp lệ trước khi cho phép truy cập API. Cần kiểm tra chữ ký, thời hạn và các điều kiện cấu hình; khóa demo trong source chỉ phục vụ lab.

Nguồn: https://www.rfc-editor.org/rfc/rfc7519

16. Hangfire

Là gì? Thư viện chạy background job cho .NET, hỗ trợ lưu trạng thái công việc và xử lý công việc ngoài luồng HTTP hiện tại.

Chức năng và liên hệ: Thực hiện tác vụ nền, lịch định kỳ và theo dõi retry. Payment dùng Hangfire cho việc đọc file khoản thu, lưu trạng thái job trong lab_netmicro_jobs; file đầu vào không được cung cấp sẵn trong sáu RAR.

Nguồn: https://docs.hangfire.io/en/latest/

17. ELK

Là gì? Tên thường dùng cho bộ Elasticsearch, Logstash và Kibana: lưu/tìm dữ liệu, thu thập/chuyển đổi dữ liệu và hiển thị/phân tích.

Chức năng và liên hệ: Hỗ trợ quản lý log tập trung và quan sát hệ thống. Lab này đã dựng Elasticsearch cho nghiệp vụ tìm kiếm; Compose không chứa Logstash hoặc Kibana, nên không báo cáo đã cài trọn bộ ELK.

Nguồn: https://www.elastic.co/elastic-stack

18. CI/CD

Là gì? CI tích hợp thay đổi thường xuyên và kiểm tra bằng quy trình tự động. Continuous Delivery chuẩn bị bản có thể phát hành; Continuous Deployment tự động phát hành khi đạt điều kiện.

Chức năng và liên hệ: Giảm lỗi tích hợp, tạo build/test lặp lại được và quản lý việc phát hành. Bài nộp có workflow build/test cơ bản; thành công ở máy local và thành công của lần chạy GitHub Actions là hai bằng chứng khác nhau.

Nguồn: https://learn.microsoft.com/en-us/devops/deliver/what-is-continuous-delivery

19. Azure DevOps

Là gì? Bộ dịch vụ hỗ trợ quản lý vòng đời phát triển phần mềm, gồm quản lý công việc, repository, pipeline, kiểm thử và artifact.

Chức năng và liên hệ: Tổ chức backlog, cộng tác source, tự động build/test và phát hành. Đề yêu cầu GitHub cho bài nộp; không cần tạo Azure DevOps project chỉ để giải thích đối tượng này.

Nguồn: https://learn.microsoft.com/en-us/azure/devops/user-guide/what-is-azure-devops

20. Azure

Là gì? Nền tảng điện toán đám mây của Microsoft, cung cấp dịch vụ tính toán, lưu trữ, mạng, dữ liệu và quản lý ứng dụng.

Chức năng và liên hệ: Có thể cung cấp hạ tầng triển khai microservices, database và giám sát theo nhu cầu hệ thống. Bài này chạy local Windows; không báo cáo đã triển khai lên Azure khi chưa tạo và kiểm tra tài nguyên cloud.

Nguồn: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-azure

21. Kubernetes

Là gì? Nền tảng điều phối workload dạng container, tổ chức workload qua các tài nguyên như Pod, Deployment và Service.

Chức năng và liên hệ: Quản lý triển khai, mở rộng và phục hồi workload theo cấu hình; hỗ trợ khám phá dịch vụ và cân bằng tải trong cluster. Docker Compose local không phải Kubernetes; bài nộp không yêu cầu dựng cluster mới.

Nguồn: https://kubernetes.io/docs/concepts/overview/